

ESE 5370: Final Project Report

Parth Sarkar

Monday, April 20th, 2026

1 Introduction

Memory safety is a key priority for programs written in C. The burden of programing defensively against malicious use of undefined behavior lies squarely on the programmer, leading to mistakes that can be exploited by user input to the program. While the problem is pervasive and fundamentally a consequence of C semantics, the Softbound+CETS project offers the strong, formally-verified guarantee of *full* memory safety. Softbound+CETS instruments C code to create and update metadata for each pointer in the program at runtime. Upon a dereference for a pointer, the pointer's metadata is cross-referenced to ensure that the dereference occurs “within bounds” and is not a use-after-free.

However, this approach has two limitations. First, from a performance perspective, it is highly expensive to branch and check any condition on every dereference; [INSERT (revisited)] cites a 140% overhead on the SPEC CPU 2017 C benchmark suite. However, more pressingly, Softbound+CETS does not offer binary-level compatibility. Hence, if any C code that has not been compiled with Softbound+CETS instrumentation is linked into a protected project, the entire executable is susceptible to memory privilege escalation.

With these problems in mind, my final project first attempts to concretize the problem of privilege escalation due to foreign-linked code with a working attack that permits a buffer-overflow write (one that would normally be stopped by Softbound+CETS instrumentation if compiled with protection). As a proposed solution, I implement and compare two encryption methods on the metadata table that Softbound+CETS uses to propagate metadata on pointers stored into memory: a simple XOR encryption, and a more complicated AES encryption method. On the performance front, I implement and formally prove important properties of a global dataflow analysis in LLVM 12 and the Rocq interactive theorem prover, respectively. I integrate the results of this analysis into the Softbound+CETS pass and demonstrate correctness and tangible performance improvements. Ultimately, I hope my project makes the case that dynamic enforcement with strong security guarantees can go hand-in-hand with provably correct, classical static analysis methods, so the security-performance tradeoff can continue to be bridged.